**TRƯỜNG ĐẠI HỌC SÀI GÒN**

**KHOA CÔNG NGHỆ THÔNG TIN**

**--------***--------**

# BÁO CÁO MÔN HỌC

# CƠ SỞ TRÍ TUỆ NHÂN TẠO

## TÊN ĐỀ TÀI

# TIỀN XỬ LÝ DỮ LIỆU VỀ BỆNH TIỂU ĐƯỜNG

**Giảng viên hướng dẫn: ThS. Đỗ Như Tài**

**Nhóm sinh viên thực hiện:**

| | |
|---|---|
| Văn Nguyễn Đức Thịnh | 3124410344 |
| Phạm Quốc Tân | 3124410315 |
| Lê Đăng Khoa | 3124410158 |
| Trần Gia Bảo | 3124410019 |
| Lê Quốc Nam | 3124410219 |

***Thành phố Hồ Chí Minh, tháng 04 năm 2026***

# Phân công

| MSSV | Họ tên | Công việc |
|---|---|---|
| 3124410344 | Văn Nguyễn Đức Thịnh | Thí nghiệm và đánh giá 6 mô hình (Lượng giá thuật toán) |
| 3124410315 | Phạm Quốc Tân | Tinh chỉnh tham số mô hình và huấn luyện mô hình trên K-fold (Lượng giá thuật toán) |
| 3124410158 | Lê Đăng Khoa | Phân chia dữ liệu và chiến lược Hold-out nội bộ (Dữ liệu kiểm nghiệm) |
| 3124410019 | Trần Gia Bảo | Kiểm nghiệm kết quả trên tập kiểm thử |
| 3124410219 | Lê Quốc Nam | Xây dựng chiến lược K-fold Cross Validation và tổng kết vai trò quy trình kiểm nghiệm (Dữ liệu kiểm nghiệm) |

# Mục lục

DỮ LIỆU KIỂM NGHIỆM ..... 4
1. Phân chia tập dữ liệu tổng thể (Global Hold-out 7/3) ..... 4
2. Chiến lược huấn luyện và điều chỉnh tham số nội bộ ..... 5
2.1. Chiến lược Hold-out nội bộ (Train/Valid Split) ..... 6
2.2. Chiến lược K-fold Cross Validation ..... 7
2.3. Tổng kết vai trò quy trình kiểm nghiệm ..... 7
LƯỢNG GIÁ THUẬT TOÁN ..... 9
1. Giới thiệu mô hình máy học ..... 9
1.1. Logistic Regression ..... 9
1.2. Linear Discriminant Analysis ..... 9
1.3. K-Nearest Neighbors ..... 9
1.4. Decision Tree ..... 10
1.5. Gaussian Naive Bayes ..... 10
1.6. Support Vector Machine ..... 10
2. Thực nghiệm mô hình máy học trên dữ liệu ..... 11
2.1. Đánh giá mô hình ..... 11
2.2. Tinh chỉnh tham số mô hình ..... 12
2.3. Huấn luyện mô hình trên K – FOLD ..... 15
2.4. Đánh giá tổng quát các mô hình từ các lần thử k - fold ..... 16
KIỂM NGHIỆM KẾT QUẢ TRÊN TẬP KIỂM THỬ ..... 18
1. Mục tiêu và chiến lược kiểm nghiệm ..... 18
2. Quy trình thực nghiệm và các độ đo đánh giá ..... 18
3. Kết quả đánh giá chung cuộc và ý nghĩa lâm sàng ..... 19
4. Đóng gói và định hướng triển khai ..... 20

# DỮ LIỆU KIỂM NGHIỆM

Trong bài toán dự đoán bệnh đái tháo đường trên tập dữ liệu Pima Indians, việc thiết lập một chiến lược phân chia dữ liệu khoa học là yêu cầu bắt buộc để đánh giá chính xác hiệu năng của mô hình phân lớp. Mục tiêu của phân đoạn này là ngăn chặn hiện tượng quá khớp (overfitting) và rò rỉ dữ liệu (data leakage), đồng thời đảm bảo mô hình có khả năng tổng quát hóa tốt trên các dữ liệu bệnh nhân chưa từng gặp.

```
print("\n=== ĐỐI CHIẾU SỐ DÒNG ===")
print(f"Số dòng ban đầu: {rows_before}")
print(f"Số dòng có lỗi (unique rows): {rows_with_error}")
print(f"Số dòng sau khi làm sạch: {rows_after}")
print(f"Số dòng bị loại thực tế: {rows_before - rows_after}")
```

```
=== ĐỐI CHIẾU SỐ DÒNG ===
Số dòng ban đầu: 768
Số dòng có lỗi (unique rows): 48
Số dòng sau khi làm sạch: 720
Số dòng bị loại thực tế: 48
```

Hình 6: Số liệu sau làm sạch

Ở đây, sau khi sàng lọc và làm sạch dữ liệu chúng ta áp dụng phương pháp chuẩn bị dữ liệu kiểm nghiệm theo đa chiến lược: **Hold-out** và **K-fold Cross-Validation**.

## 1. Phân chia tập dữ liệu tổng thể (Global Hold-out 7/3)

Nếu hệ thống tính toán sử dụng trực tiếp tập Test 238 mẫu vừa bị đóng băng để làm cơ sở tinh chỉnh các siêu tham số này, mô hình sẽ dần "học vẹt" (overfit) các vi cấu trúc đặc tính của chính tập Test đó.

Do đó, hệ thống dữ liệu trong tập Train (482 mẫu) buộc phải được cắt nhỏ thêm một cấp độ phân rã nữa thông qua các chiến lược lượng giá nội bộ: Hold-out phân cấp và Kiểm chứng chéo K-Fold.

Tập dữ liệu gốc sau khi trải qua bước tiền xử lý và làm sạch được phân chia thành hai phần độc lập theo tỉ lệ **7/3** (70% cho huấn luyện và 30% cho kiểm thử).

- **Tập Train (Huấn luyện):** Chứa **482 mẫu** (tương ứng file `train.xlsx - Sheet1.csv`). Tập này được sử dụng để thuật toán học các trọng số, đặc trưng sinh lý và tinh chỉnh siêu tham số.

- **Tập Test (Kiểm nghiệm):** Chứa **238 mẫu** (tương ứng file `test.xlsx - Sheet1.csv`). Tập này bị "đóng băng" hoàn toàn trong suốt quá trình xây dựng mô hình và chỉ được sử dụng ở bước đánh giá cuối cùng nhằm đo lường tính hiệu quả thực tế của thuật toán.

### Chia dữ liệu thực nghiệm

- Dữ liệu ban đầu: chia 30% dữ liệu dùng để Test, 70% dùng để train
- Dữ liệu train: chia chiến lược K-Fold với k = 5

```
index_all = np.arange(len(df_org))
index_train, index_test = train_test_split(index_all, test_size = 0.33, random_state=seed)
```

```
df_train = df_org.iloc[index_train]
df_test  = df_org.iloc[index_test]
print(f'train: {len(df_train)} / test: {len(df_test)} - Tỷ lệ: {len(df_train) / len(df_org)}, {len(df_test) / len(df_org)}')
```

```
train: 514 / test: 254 - Tỷ lệ: 0.6692708333333334, 0.3307291666666667
```

Hình 6.1.1: Chia dữ liệu thực nghiệm


### Chia dữ liệu thực nghiệm

- Dữ liệu ban đầu: chia 30% dữ liệu dùng để Test, 70% dùng để train
- Dữ liệu train: chia chiến lược K-Fold với k = 5

```
index_all = np.arange(len(df_org))
index_train, index_test = train_test_split(index_all, test_size = 0.33, random_state=seed)
```

```
df_train = df_org.iloc[index_train]
df_test  = df_org.iloc[index_test]
print(f'train: {len(df_train)} / test: {len(df_test)} - Tỷ lệ: {len(df_train) / len(df_org)}, {len(df_test) / len(df_org)}')
```


```
train: 482 / test: 238 - Tỷ lệ: 0.6694444444444444, 0.33055555555555555
```

Hình 6.1.2: Dữ liệu thực hiện sau làm sạch

## 2. Chiến lược huấn luyện và điều chỉnh tham số nội bộ

Để đánh giá và tinh chỉnh tham số (Hyperparameter tuning) cho các mô hình (như SVM, K-Neighbors, Decision Tree,...) mà không làm rò rỉ thông tin từ tập Test, tập Train (482 mẫu) buộc phải được cắt nhỏ thêm một cấp độ phân rã nữa thông qua các chiến lược lượng giá nội bộ: Hold-out phân cấp và Kiểm chứng chéo K-Fold.

## 2.1. Chiến lược Hold-out nội bộ (Train/Valid Split)

Tập Train (514 mẫu) tiếp tục được chia nhỏ theo tỉ lệ **7/3** để tạo ra một tập kiểm định (Validation set) chuyên biệt. Sau quá trình xử lý nhiễu nội bộ, cấu trúc tập dữ liệu thực tế thu được lưu trong file `holdout_idx.npz` như sau:

- **Tập Train nội bộ (Train sub-set): 337 mẫu** (dùng để khớp mô hình).
- **Tập Validation (Valid set): 145 mẫu** (dùng để đánh giá và hiệu chỉnh tham số nhanh chóng).
- Tổng cộng có 482 mẫu (lọc đi 32 hồ sơ).

```python
# Tách feature và label

X_train = df_train.iloc[:, :-1]
y_train = df_train.iloc[:, -1]

X_test  = df_test.iloc[:, :-1]
y_test  = df_test.iloc[:, -1]
```

```python
scaler = StandardScaler()

x_train = scaler.fit_transform(X_train)
x_test  = scaler.transform(X_test)
y_train = y_train.values
```

```python
X_train_hold, X_valid_hold, y_train_hold, y_valid_hold = train_test_split(
    x_train,
    y_train,
    test_size=0.3,
    random_state=seed,
    stratify=y_train
)

print("Hold-out split")
print("Train:", X_train_hold.shape)
print("Valid:", X_valid_hold.shape)
```

```
Hold-out split
Train: (337, 8)
Valid: (145, 8)
```

Hình 6.2.1: Mã nguồn và kết quả chiến lược Hold-out nội bộ

Tuy nhiên, dù tốc độ thực thi rất nhanh, chiến lược Hold-out nội bộ này tồn tại nhược điểm toán học nghiêm trọng: phương sai cao (high variance) trong đánh giá hiệu năng nếu các mẫu bệnh lý y khoa phức tạp tình cờ lọt hết vào tập Valid hoặc ngược lại.

## 2.2. Chiến lược K-fold Cross Validation

Mặc dù Hold-out nội bộ cho tốc độ tính toán nhanh, nhưng nó dễ bị ảnh hưởng bởi phương sai dữ liệu nếu cách chia ngẫu nhiên không hội tụ đủ các trường hợp y khoa phức tạp. Để khắc phục, chiến lược **K-fold (với k=5 hoặc 10)** được áp dụng trên toàn bộ tập Train gốc.

Phương pháp này chia dữ liệu huấn luyện thành k phần (folds) đều nhau. Quá trình mô hình hóa sẽ lặp lại k lần, mỗi lần lấy k-1 phần làm dữ liệu Train và 1 phần còn lại làm Validation. Điểm số đánh giá cuối cùng là trung bình cộng của k lần kiểm định.

**k-fold (chia thành k phần đều nhau với k-1 phần cho train/1 phần cho valid)**

```python
kfold = StratifiedKFold(
    n_splits=10,
    shuffle=True,
    random_state=seed
)

print(kfold)
```

```
StratifiedKFold(n_splits=10, random_state=42, shuffle=True)
```

```python
for train_idx, valid_idx in kfold.split(x_train, y_train):

    X_train_k = x_train[train_idx]
    y_train_k = y_train[train_idx]

    X_valid_k = x_train[valid_idx]
    y_valid_k = y_train[valid_idx]

    print("Train:", X_train_k.shape, "Valid:", X_valid_k.shape)
    break
```

```
Train: (433, 8) Valid: (49, 8)
```

Hình 6.2.2: Mã nguồn và kết quả thực thi thiết lập K-fold

## 2.3. Tổng kết vai trò quy trình kiểm nghiệm

Sự kết hợp giữa chia tách tổng thể (`idx.npz`) và chia tách nội bộ (`holdout_idx.npz`, `K-fold`) tạo thành một chu trình khép kín:

1. **Train set (k-1 folds hoặc 70% nội bộ):** Môi trường để thuật toán tìm ra quy luật từ đặc trưng sinh lý học (như mức đường huyết, độ dày biểu bì, BMI,...).
2. **Valid set (1 fold hoặc 30% nội bộ):** Thước đo tạm thời để tìm ra bộ tham số (parameters) mang lại độ chính xác, độ nhạy cao nhất.

3. **Test set (254 mẫu độc lập):** Giám khảo cuối cùng. Kết quả đánh giá trên tập này là kết quả chính thức được báo cáo trong ứng dụng chẩn đoán y khoa thực tế (Mục 8). Quy trình này tuân thủ tuyệt đối tiêu chuẩn đánh giá thuật toán máy học hiện đại.
4. **Tính sẵn sàng:** Dữ liệu đã được đóng gói (feat_minmax.npz, feat_standard.npz) và chuẩn bị đầy đủ cho Bước 7 (Lượng giá thuật toán) và Bước 8 (Kiểm nghiệm kết quả) một cách trơn tru.
5. **Tính tái lập (Reproducibility):** Toàn bộ trạng thái phân chia của thí nghiệm được lưu trữ chuẩn xác thông qua các ma trận Numpy (idx.npz, holdout_idx.npz), cho phép các nhà nghiên cứu khác có thể dễ dàng kiểm chứng chéo lại kết quả của dự án.

# LƯỢNG GIÁ THUẬT TOÁN

## 1. Giới thiệu mô hình máy học

Để thực hiện bài toán phân loại bệnh tiểu đường, một số thuật toán học máy phổ biến được lựa chọn làm mô hình cơ sở (baseline). Các mô hình này đại diện cho nhiều cách tiếp cận khác nhau trong học máy, từ tuyến tính, dựa trên khoảng cách, đến dựa trên xác suất và siêu phẳng phân tách. Việc sử dụng đa dạng mô hình giúp so sánh và lựa chọn thuật toán phù hợp nhất với dữ liệu.

### 1.1. Logistic Regression

Logistic Regression (LR) là một thuật toán phân loại tuyến tính được sử dụng phổ biến cho các bài toán phân loại nhị phân. Mô hình hoạt động bằng cách xây dựng một hàm tuyến tính từ các đặc trưng đầu vào, sau đó đưa kết quả qua hàm "sigmoid" để chuyển đổi thành xác suất thuộc về một lớp cụ thể. Trong quá trình huấn luyện, mô hình sử dụng tập train để học các trọng số tối ưu, nhằm tối thiểu hóa sai số dự đoán. Khi áp dụng lên tập test, mô hình sử dụng các trọng số đã học để tính xác suất và gán nhãn lớp dự đoán tương ứng.

### 1.2. Linear Discriminant Analysis

Linear Discriminant Analysis (LDA) là phương pháp phân loại dựa trên giả định dữ liệu của mỗi lớp tuân theo phân phối chuẩn và có cùng ma trận hiệp phương sai. Thuật toán tìm một không gian chiếu tuyến tính sao cho khoảng cách giữa các lớp được tối đa hóa và phương sai trong từng lớp được tối thiểu hóa. Trong giai đoạn huấn luyện, LDA ước lượng các tham số phân phối từ tập train. Khi dự đoán trên tập test, mô hình sử dụng các tham số đã học để xác định xác suất một mẫu thuộc về từng lớp và chọn lớp có xác suất cao nhất.

### 1.3. K-Nearest Neighbors

K-Nearest Neighbors là thuật toán phân loại dựa trên khoảng cách, không xây dựng mô hình tường minh trong quá trình huấn luyện. Thay vào đó, toàn bộ dữ liệu train được lưu lại. Khi dự đoán một mẫu mới từ tập test, thuật toán tính khoảng cách từ mẫu đó đến tất cả các mẫu trong tập train, sau đó chọn K mẫu gần nhất và quyết định nhãn theo đa số. Phương pháp này đơn giản nhưng hiệu quả, đặc biệt khi dữ liệu đã được chuẩn hóa.

### 1.4. Decision Tree

Decision Tree là thuật toán phân loại dựa trên cấu trúc cây quyết định. Mô hình hoạt động bằng cách chia dữ liệu train thành các nhánh dựa trên các điều kiện của đặc trưng, nhằm tăng độ thuần khiết của các nút. Quá trình huấn luyện sẽ xây dựng cây từ gốc đến lá bằng cách chọn thuộc tính phân chia tốt nhất tại mỗi bước. Khi dự đoán trên tập test, mỗi mẫu sẽ đi theo các nhánh điều kiện cho đến khi đến nút lá và nhận nhãn tương ứng.

### 1.5. Gaussian Naive Bayes

Gaussian Naive Bayes là thuật toán phân loại dựa trên định lý Bayes với giả định các đặc trưng độc lập với nhau và tuân theo phân phối chuẩn. Trong quá trình huấn luyện, mô hình tính toán xác suất tiên nghiệm của từng lớp và các tham số phân phối cho từng đặc trưng từ tập train. Khi dự đoán trên tập test, thuật toán sử dụng công thức Bayes để tính xác suất hậu nghiệm của từng lớp và chọn lớp có xác suất cao nhất.

### 1.6. Support Vector Machine

Support Vector Machine là thuật toán phân loại dựa trên việc tìm siêu phẳng tối ưu để phân tách các lớp dữ liệu. Mô hình cố gắng tối đa hóa khoảng cách giữa siêu phẳng và các điểm dữ liệu gần nhất (support vectors). Trong quá trình huấn luyện, SVM học từ tập train để xác định siêu phẳng phân tách tốt nhất. Khi dự đoán trên tập test, các mẫu mới sẽ được xác định nằm ở phía nào của siêu phẳng và được gán nhãn tương ứng. SVM có thể sử dụng các kernel để xử lý các trường hợp dữ liệu không phân tách tuyến tính.

## 2. Thực nghiệm mô hình máy học trên dữ liệu

### 2.1. Đánh giá mô hình

```
LR
Accuracy: 0.7720663265306122
Std: 0.04726179843101965
----------------
LDA
Accuracy: 0.7721513605442176
Std: 0.041005641774539366
----------------
KNN
Accuracy: 0.7477891156462585
Std: 0.0716473508091368
----------------
CART
Accuracy: 0.718579931972789
Std: 0.09299013713930986
----------------
NB
Accuracy: 0.753656462585034
Std: 0.056858513172203494
----------------
SVM
Accuracy: 0.7740646258503401
Std: 0.045914902867541164
----------------
```

*Bảng đánh giá độ chính xác của từng mô hình*

Kết quả đánh giá cho thấy các mô hình tuyến tính như Logistic Regression và Linear Discriminant Analysis đạt độ chính xác trung bình tương đối cao và ổn định.

→ Phân tách khá tốt trong không gian tuyến tính.

Mô hình Support Vector Machine cũng đạt hiệu quả tương đương và có độ ổn định tốt, thể hiện qua độ lệch chuẩn nhỏ.

→ Tổng quát hóa tốt, ít bị ảnh hưởng bởi thay đổi của tập dữ liệu.

Mô hình K-Nearest Neighbors đạt độ chính xác thấp hơn một chút so với các mô hình tuyến tính, đồng thời có độ lệch chuẩn lớn hơn.

→ Phụ thuộc khá nhiều vào cách phân chia dữ liệu, có thể bị ảnh hưởng nếu nhiễu loạn trong quá trình xử lý tập dữ liệu.

Mô hình Decision Tree là mô hình có độ chính xác thấp nhất trong các thuật toán được thử nghiệm và độ lệch chuẩn tương đối lớn.

→ Bị overfitting khi dữ liệu thay đổi, nhưng khả năng giải thích tốt và tốc độ huấn luyện nhanh.

Gaussian Naive Bayes cho kết quả trung bình khá và độ ổn định ở mức chấp nhận được.

→ Bị hạn chế khi các thuộc tính có mối quan hệ với nhau.

**Kết luận:** Mô hình Logistic Regression, Linear Discriminant Analysis và Support Vector Machine cho kết quả tốt và ổn định hơn so với các thuật toán còn lại. Trong đó, Support Vector Machine đạt độ chính xác cao nhất và độ lệch chuẩn nhỏ, cho thấy khả năng tổng quát hóa tốt nhất trên tập dữ liệu.

## 2.2. Tinh chỉnh tham số mô hình

Trong thực nghiệm này, hai thuật toán được lựa chọn để tối ưu là K-Nearest Neighbors và Support Vector Machine, do hiệu suất của các mô hình này phụ thuộc nhiều vào tham số cấu hình.

### *2.2.1.* Tinh chỉnh tham số mô hình K-Nearest Neighbors

Sau khi đánh giá mô hình K-Nearest Neighbors với tham số mặc định, bước tiếp theo là tiến hành tinh chỉnh tham số nhằm cải thiện hiệu suất phân loại. Trong thuật toán KNN, tham số quan trọng nhất là số lượng láng giềng gần nhất K. Giá trị của K ảnh hưởng trực tiếp đến khả năng học của mô hình. Nếu K quá nhỏ, mô hình dễ bị ảnh hưởng bởi nhiễu; nếu K quá lớn, mô hình có thể làm mất đi ranh giới phân tách giữa các lớp.

Để tìm giá trị K tối ưu, phương pháp tìm kiếm trên lưới (Grid Search) được sử dụng kết hợp với K-Fold Cross Validation. Nhiều giá trị của tham số K được thử nghiệm, sau đó mô hình được huấn luyện và đánh giá trên từng cấu hình. Giá trị tham số cho kết quả độ chính xác trung bình cao nhất được lựa chọn làm cấu hình tối ưu.

```
param_grid = {
"n_neighbors": [3,5,7,9,11]
}

grid = GridSearchCV(
KNeighborsClassifier(),
param_grid,
cv=kfold,
scoring="accuracy"
)

grid.fit(x_train, y_train)

print("Best score:", grid.best_score_)
print("Best params:", grid.best_params_)
```

*Chương trình tinh chỉnh mô hình K-Nearest Neighbors*

```
Best score: 0.7763605442176871

Best params: {'n_neighbors': 7}
```

*Kết quả tinh chỉnh mô hình K-Nearest Neighbors*

Trước khi tinh chỉnh: độ chính xác ≈ 0.748.

Sau khi tỉnh chỉnh, độ chính xác ≈ 0.776.

Kết quả tinh chỉnh cho thấy mô hình KNN sau khi lựa chọn tham số phù hợp đạt hiệu suất tốt hơn so với cấu hình mặc định.

### *2.2.2.* Tinh chỉnh tham số mô hình Support Vector Machine

Đối với Support Vector Machine, các tham số như hệ số phạt C và loại kernel có ảnh hưởng lớn đến khả năng phân tách dữ liệu. Tham số C điều chỉnh mức độ cho phép sai số trong quá trình huấn luyện, trong khi kernel giúp mô hình xử lý các trường hợp dữ liệu không phân tách tuyến tính. Việc thử nghiệm nhiều cấu hình khác nhau giúp tìm ra mô hình có hiệu suất tốt nhất.

Sau khi đánh giá các mô hình với tham số mặc định, thuật toán Support Vector Machine cũng được tiến hành tinh chỉnh tham số nhằm cải thiện hiệu quả phân loại. Hiệu suất của SVM phụ thuộc nhiều vào các tham số điều chỉnh, đặc biệt là hệ số phạt C, loại kernel và tham số gamma.

Phương pháp Grid Search kết hợp với K-Fold Cross Validation được sử dụng. Nhiều tổ hợp tham số điều chỉnh được thử nghiệm, mô hình được huấn luyện và đánh giá trên từng cấu hình. Tổ hợp tham số có độ chính xác trung bình cao nhất được lựa chọn làm cấu hình tối ưu cho SVM.

```
param_grid = {
    "C": [0.1, 1, 10, 100],
    "kernel": ["linear", "rbf"],
    "gamma": ["scale", "auto"]
}

grid = GridSearchCV(
    SVC(),
    param_grid,
    cv=kfold,
    scoring="accuracy"
)

grid.fit(x_train, y_train)

print("Best score:", grid.best_score_)
print("Best params:", grid.best_params_)
```

*Chương trình tinh chỉnh mô hình Support Vector Machine*

```
Best score: 0.7762755102040816

Best params: {'C': 0.1, 'gamma': 'scale', 'kernel': 'linear'}
```

*Kết quả tinh chỉnh mô hình Support Vector Machine*

Trước khi tinh chỉnh: Độ chính xác ≈ 0.774.

Sau khi tinh chỉnh: Độ chính xác ≈ 0.776.

Kết quả tinh chỉnh cho thấy mô hình Support Vector Machine sau khi tối ưu tham số đạt độ chính xác cao và ổn định hơn. Điều này chứng tỏ việc lựa chọn phù hợp các tham số C, kernel và gamma giúp mô hình khai thác tốt hơn cấu trúc của dữ liệu và cải thiện khả năng phân loại.

## 2.3. Huấn luyện mô hình trên K – FOLD

Trong quá trình xây dựng và đánh giá mô hình học máy, một trong những thách thức lớn là đảm bảo kết quả đánh giá phản ánh đúng khả năng tổng quát hóa của mô hình trên dữ liệu chưa từng xuất hiện. Nếu chỉ sử dụng một lần chia dữ liệu thành tập huấn luyện và tập kiểm tra, kết quả thu được có thể bị phụ thuộc mạnh vào cách phân chia này, từ đó dẫn đến đánh giá thiếu khách quan hoặc không ổn định. Để khắc phục vấn đề đó, đề tài áp dụng phương pháp k-fold cross validation trong quá trình huấn luyện và đánh giá mô hình.

Cụ thể, toàn bộ tập dữ liệu ban đầu được chia thành $k$ phần (gọi là các "fold") có kích thước xấp xỉ bằng nhau. Giá trị $k$ thường được lựa chọn là 5 hoặc 10 nhằm đảm bảo sự cân bằng giữa độ chính xác của đánh giá và chi phí tính toán. Sau khi chia dữ liệu, quá trình huấn luyện và kiểm tra được thực hiện lặp lại $k$ lần. Ở mỗi lần lặp, một fold sẽ được sử dụng làm tập kiểm tra (validation set), trong khi $k - 1$ fold còn lại được sử dụng làm tập huấn luyện.

**k-fold (chia thành k phần đều nhau với k-1 phần cho train/1 phần cho valid)**

```
kfold = StratifiedKFold(
    n_splits=10,
    shuffle=True,
    random_state=seed
)

print(kfold)
```

Sau khi hoàn tất quá trình huấn luyện, mỗi mô hình sẽ thu được $k$ kết quả đánh giá tương ứng với từng lần lặp. Các kết quả này đã được tổng hợp bằng cách tính giá trị trung bình nhằm đưa ra đánh giá cuối cùng cho từng mô hình. Nhờ đó, hiệu suất của mô hình không còn phụ thuộc vào một cách chia dữ liệu duy nhất mà phản ánh chính xác hơn khả năng dự đoán trên toàn bộ tập dữ liệu.

```
results = []
names = []

for name, model in models:

    scores = cross_val_score(
        model,
        x_train,
        y_train,
        cv=kfold,
        scoring="accuracy"
    )


    results.append(scores)
    names.append(name)

    print(name)
    print("Accuracy:", scores.mean())
    print("Std:", scores.std())
    print("----------------")
```

Từ các kết quả thực nghiệm sau mỗi lần chạy cho thấy mô hình đạt kết quả ổn định tương đối cao giữa mỗi lần lặp.

Dựa trên bảng đánh giá mô hình ở trên từ đó giúp lựa chọn được mô hình có hiệu suất tối ưu

Tóm lại trong quá trình chạy thử, và thử nghiệm việc áp dụng. Tóm lại việc áp dụng k-fold cross validation đã góp phần nâng cao độ tin cậy trong quá trình huấn luyện và đánh giá mô hình, đồng thời đảm bảo rằng mô hình được lựa chọn có khả năng hoạt động tốt trên dữ liệu thực tế trong bài toán dự đoán bệnh tiểu đường.

### 2.4. Đánh giá tổng quát các mô hình từ các lần thử k - fold

```
LR _tuned
Accuracy: 0.7782312925170067
Std: 0.038572182172388336
----------------
LDA _tuned
Accuracy: 0.7721513605442176
Std: 0.041005641774539366
----------------
KNN _tuned
Accuracy: 0.7803146258503401
Std: 0.05479452552839471
----------------
CART _tuned
Accuracy: 0.7454931972789116
Std: 0.07216357344153954
----------------
NB _tuned
Accuracy: 0.753656462585034
Std: 0.056858513172203494
----------------
SVM _tuned
Accuracy: 0.7802721088435374
Std: 0.04298122424883498
----------------
```

Sau khi tinh chỉnh tham số bằng phương pháp k-fold cross validation, có thể tiến hành đánh giá các mô hình dựa trên hai tiêu chí chính là độ chính xác trung bình (accuracy) và độ lệch chuẩn (standard deviation) như sau:

Kết quả cho thấy mô hình Support Vector Machine (SVM) đạt hiệu suất tốt nhất với độ chính xác trung bình khoảng 0.7803 và độ lệch chuẩn khoảng 0.0430. Đây

là mô hình có độ chính xác cao nhất trong số các mô hình được thử nghiệm, đồng thời độ lệch chuẩn tương đối thấp cho thấy tính ổn định tốt khi thay đổi các tập dữ liệu huấn luyện. Điều này cho thấy SVM là lựa chọn phù hợp cho bài toán, đặc biệt trong trường hợp dữ liệu có thể tồn tại các mối quan hệ phi tuyến.

Mô hình Logistic Regression (LR) đạt độ chính xác trung bình khoảng 0.7782 với độ lệch chuẩn khoảng 0.0386. Mặc dù độ chính xác thấp hơn SVM một chút, nhưng LR lại có độ lệch chuẩn thấp nhất trong tất cả các mô hình, cho thấy đây là mô hình ổn định nhất. Điều này chứng tỏ LR vẫn là một lựa chọn rất đáng tin cậy, đặc biệt khi ưu tiên tính nhất quán của mô hình.

Tiếp theo, mô hình Linear Discriminant Analysis (LDA) đạt độ chính xác khoảng 0.7722 và độ lệch chuẩn khoảng 0.0410. Hiệu suất của LDA khá tương đồng với LR, tuy nhiên độ ổn định kém hơn một chút. Dù vậy, đây vẫn là một mô hình tuyến tính cho kết quả tốt và có thể được cân nhắc sử dụng.

Đối với Naive Bayes (NB), mô hình đạt độ chính xác trung bình khoảng 0.7537 với độ lệch chuẩn khoảng 0.0569. Kết quả này ở mức trung bình, và độ lệch chuẩn tương đối cao cho thấy mô hình chưa thực sự ổn định. Nguyên nhân có thể đến từ giả định độc lập giữa các đặc trưng không hoàn toàn phù hợp với dữ liệu.

Mô hình K-Nearest Neighbors (KNN) đạt độ chính xác khoảng 0.7803 nhưng lại có độ lệch chuẩn cao hơn (~0.0548), cho thấy sự biến động lớn giữa các lần huấn luyện. Mặc dù độ chính xác trung bình tương đối cao, nhưng tính ổn định chưa tốt khiến mô hình này kém tin cậy hơn so với SVM và LR.

Cuối cùng, mô hình Decision Tree (CART) cho kết quả thấp nhất với độ chính xác khoảng 0.7455 và độ lệch chuẩn khoảng 0.0722. Đây là mô hình có độ ổn định kém nhất, thể hiện qua độ lệch chuẩn cao, cho thấy khả năng overfitting và nhạy cảm với dữ liệu.

**Tổng kết lại**, mô hình SVM là lựa chọn tốt nhất xét về cả độ chính xác và độ ổn định. Logistic Regression cũng là một phương án rất đáng cân nhắc nhờ tính ổn định cao. Các mô hình còn lại như LDA, NB, KNN và CART có thể cần được cải thiện thêm để đạt hiệu quả tốt hơn trong bài toán.

# KIỂM NGHIỆM KẾT QUẢ TRÊN TẬP KIỂM THỬ

## 1. Mục tiêu và chiến lược kiểm nghiệm

Trong vòng đời của một dự án Khoa học dữ liệu, giai đoạn mang tính quyết định và phản ánh đúng nhất năng lực thực tiễn của hệ thống chính là bước kiểm nghiệm mô hình trên tập dữ liệu chưa từng quan sát. Mục tiêu cốt lõi của kiểm nghiệm không chỉ dừng lại ở việc so sánh các con số thống kê khô khan, mà là để kiểm chứng khả năng tổng quát hóa của các thuật toán, đảm bảo rằng mô hình không mắc phải hiện tượng học vẹt từ tập huấn luyện.

Để hiện thực hóa mục tiêu này, 30% dữ liệu (tương đương 216 bản ghi) đã được nhóm nghiên cứu chủ động cách ly tuyệt đối ngay từ những bước đầu tiên của quy trình. Chiến lược kiểm nghiệm được thiết kế theo hướng đối chiếu song song: một mặt tiến hành lượng giá sáu mô hình cơ sở bao gồm Hồi quy Logistic, LDA, KNN, Cây quyết định, Naive Bayes và SVM để thiết lập tiêu chuẩn tối thiểu; mặt khác, đưa hai mô hình đã được tinh chỉnh siêu tham số chuyên sâu (KNN và SVM đã tinh chỉnh) vào tham chiến. Thông qua phép thử này, chúng ta có thể đánh giá được chính xác mức độ hiệu quả của việc dò tìm tham số tác động như thế nào đến độ sắc nét của ranh giới quyết định lâm sàng.

## 2. Quy trình thực nghiệm và các độ đo đánh giá

Nhằm đảm bảo tính nhất quán tuyệt đối với các phát hiện từ quá trình Phân tích Khám phá, tập dữ liệu đầu vào (gồm 720 bản ghi đã được làm sạch hoàn toàn các giá trị sinh lý phi logic) được phân chia theo tỷ lệ 7/3. Đặc biệt, cơ chế lấy mẫu phân tầng được áp dụng nghiêm ngặt nhằm duy trì tỷ lệ hiện mắc bệnh ở mức 34.44% đồng đều trên cả tập huấn luyện và tập kiểm thử. Việc này giúp các độ đo đánh giá sau cùng phản ánh đúng xác suất rủi ro của quần thể Pima Indians ngoài thực tế.

Trong bước tiền xử lý cuối cùng, toàn bộ tập kiểm thử được đưa qua bộ chuyển đổi phân phối chuẩn. Một nguyên tắc bất di bất dịch trong học máy đã được tuân thủ: bộ chuyển đổi này chỉ được "khớp" trên tập huấn luyện và chỉ thực hiện "biến đổi" trên tập kiểm thử. Thao tác này giúp triệt tiêu hoàn toàn rủi ro rò rỉ dữ liệu – một lỗi sai chí mạng có thể dẫn đến việc đánh giá sai lệch hiệu năng thực sự của thuật toán. Cuối cùng, tám biến thể mô hình lần lượt được gọi hàm dự đoán và đo lường

thông qua một hệ thống các ma trận đánh giá toàn diện bao gồm: Độ chính xác tổng thể, Độ chụm, Độ nhạy và Điểm F1.

## 3. Kết quả đánh giá chung cuộc và ý nghĩa lâm sàng

```
============================================================
 KẾT QUẢ ĐÁNH GIÁ TRÊN TẬP TEST (MỤC 8)
============================================================
 > Logistic Regression (Base)  : Accuracy = 0.7156 (71.56%)
 > LDA (Base)                  : Accuracy = 0.7248 (72.48%)
 > Decision Tree (Base)        : Accuracy = 0.6697 (66.97%)
 > Naive Bayes (Base)          : Accuracy = 0.6789 (67.89%)
 > KNN (Tuned)                 : Accuracy = 0.7339 (73.39%)
 > SVM (Tuned)                 : Accuracy = 0.7385 (73.85%)
------------------------------------------------------------
 MÔ HÌNH XUẤT SẮC NHẤT: SVM (Tuned) với Accuracy = 73.85%

[BÁO CÁO CHI TIẾT MÔ HÌNH TỐT NHẤT]
               precision    recall  f1-score   support

Khỏe mạnh (0)       0.78      0.83      0.81       143
 Mắc bệnh (1)       0.64      0.56      0.60        75
...
 weighted avg       0.73      0.74      0.73       218
```

Dựa trên kết quả từ quá trình chạy mã lệnh thực tế trên tập kiểm thử, bức tranh về năng lực của các mô hình đã được bộc lộ một cách vô cùng chân thực. Đáng chú ý nhất, mô hình Support Vector Machine (SVM) sau khi được tinh chỉnh siêu tham số đã vươn lên dẫn đầu bảng xếp hạng với độ chính xác tổng thể đạt 73.85%.

Về mặt kỹ thuật, đây là một bước tiến lớn của nhóm nghiên cứu trong việc ứng dụng không gian nhân đa thức để tối ưu hóa siêu phẳng phân tách dữ liệu y sinh đa chiều. Tuy nhiên, dưới góc độ học thuật khắt khe, kết quả thực nghiệm cũng chỉ ra một thực tế khách quan: mức hiệu năng này vẫn chưa thể xô đổ được ngưỡng lịch sử 76% mà thuật toán mạng nơ-ron thích nghi ADAP từng thiết lập vào năm 1988. Sự thừa nhận này không làm giảm đi giá trị của mô hình, mà ngược lại, nó minh chứng cho thái độ làm việc trung thực, bám sát tuyệt đối vào dữ liệu thực nghiệm của nhóm nghiên cứu.

Đi sâu vào việc phân tích ma trận nhầm lẫn, chỉ số Độ nhạy – tức tỷ lệ nhận diện đúng người thực sự mắc bệnh – của mô hình SVM hiện đang dừng lại ở mức 56%.

Dưới lăng kính của y tế công cộng, con số này bộc lộ một thách thức vô cùng khắc nghiệt. Dù độ chính xác tổng thể tương đối khả quan, hệ thống vẫn đang tiềm ẩn rủi ro bỏ lọt một lượng đáng kể bệnh nhân mang mầm bệnh (tức tỷ lệ âm tính giả còn cao). Điều này mở ra một định hướng quan trọng cho các nghiên cứu tiếp theo: hệ thống cần được tích hợp thêm các kỹ thuật phạt trọng số chênh lệch hoặc chủ động hạ ngưỡng chẩn đoán để ép thuật toán phải ưu tiên tối đa hóa Độ nhạy, nhằm đáp ứng trọn vẹn tiêu chuẩn khắt khe của một hệ thống tầm soát y tế tuyến đầu.

## 4. Đóng gói và định hướng triển khai

Một dự án học máy chỉ thực sự khép lại khi các thành quả thuật toán được chuẩn bị sẵn sàng cho việc đưa vào môi trường sản xuất. Nhằm phục vụ cho công tác định hướng chuyển giao công nghệ, cấu trúc của mô hình SVM xuất sắc nhất cùng bộ véc-tơ chuẩn hóa dữ liệu đã được nhóm xuất thành công thông qua thư viện Joblib của Python.

Việc lưu trữ cả bộ cấu trúc chuẩn hóa và mô hình đảm bảo rằng trong tương lai, khi hệ thống được nhúng vào các phần mềm Quản lý Bệnh án Điện tử tại các bệnh viện, bất kỳ một dữ liệu sinh hiệu mới nào của bệnh nhân nhập vào đều sẽ được tự động chuẩn hóa đúng tỷ lệ và đi qua màng lọc thuật toán để đưa ra cảnh báo nguy cơ.